COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/6/2026	**DATE SUBMITTED:**	10/6/2026
NAMES:	Dimaguiba, Lorenzo James M. Urbano, Vince Andrew F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

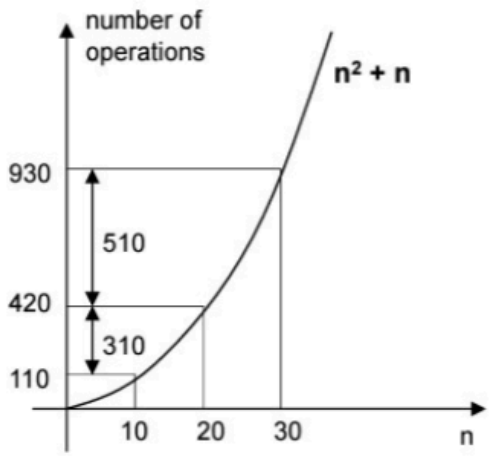
ILO A:

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:
Let m = the length of array x.
Let n = the length of array y.
The loop in diff repeats n times.
Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: O(n * m)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
Based on the given scenarios of the worst case scenario (can also be written with O(n^2)) the number of operations increases or scales depending or with the increasing size of the array with the addition of n. Meaning, after the first loop or process is complete, the value or size of the array would be added to the final number of operations. The growth shown is exponential, as seen on the graph.

ILO B:

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;
    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
    }

    // getters
    auto get_name() {
        return name;
    }
    auto is_vaccinated() {
        return vaccinated;
    }

        //two ppl are same if they have same name
    bool operator == (const Student &p) const{
        return this->name == p.name;
    }
```

```cpp
    }

    //ordering of a set of ppl is defined by their name
    bool operator < (const Student &p) const{
        return this->name < p.name;
    }

    bool operator > (const Student &p) const{
        return this->name > p.name;
    }
};

auto generate_random_Student (int max){
    std::random_device rd;
    std::mt19937 rand(rd());

    //ID of student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1,max);

    //generate rand creds
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student (random_name, is_vaccinated);
}
```

```cpp
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

    // returns true if the Student is found in the sequence and not vaccinated
    if (mid_element == P && mid_element.is_vaccinated() == false) {
        return true;
    }

    else if (mid_element == P && mid_element.is_vaccinated() == true) {
        return false;
    }

    else if (mid_element > P) {
        std::advance(last, -mid_element_index);
    }

    if (mid_element < P) {
        std::advance(first, mid_element_index);
    }

    if (range_length == 1) {
        return true;
    }
    }
}
```

Input size N	Execution Speed	Screenshot	**Analysis/Observation(s):**

1000	Dimaguiba:
	1 µsecond
	Urbano:
	0 µsecond
		Dimaguiba:
		`Time taken to search = 1microseconds`
		`Student (980 467) Needs Vaccination.`
		Urbano:
		`Time taken to search = 0microsecond`
		`Student (9084 29294) Needs Vaccinat`
			The difference is very small considering the devices or desktops in which the programs were run, have similar specifications in terms of parts, meaning similar

10000	Dimaguiba:
	1 µsecond
	Urbano:
	2 µsecond
		Dimaguiba:
		`Time taken to search = 1microseconds`
		`Student (980 467) Needs Vaccination.`
		Urbano:
		`Time taken to search = 2microseconds`
		`Student (9084 29294) Needs Vaccination.`
			performance. The program generates a random student. Whether or not they would need vaccination would depend, which code from step 3 demonstrates,

100000	Dimaguiba
	2 µsecond
	Urbano:
	2 µsecond
		Dimaguiba:
		`Time taken to search = 2microseconds`
		`Student (980 467) Needs Vaccination.`
		Urbano
		`Time taken to search = 2microseconds`
		`Student (9084 29294) Needs Vaccination.`	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

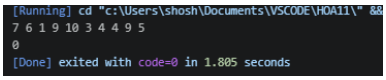
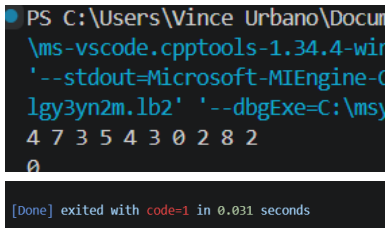
Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis
For each of the problems below, provide the following:

Theoretical Analysis
Experimental Analysis
Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

	Dimaguiba	Urbano
Theoretical Analysis	T(n) = n(n-1)/2 or n^2 - n/ 2, Worst case of: O(n^2)	Since this program is very similar to the bubble sort, T(n) = O(n^2)

Experimental Analysis		
Analysis and Comparison	I starts at 0, then (much like i<n) runs up till (n-1), another loop connected to “i”, “j” runs the same but runs each time after the “i” loop as well as its comparison returning either true or false (n - 1). Combining the processes on the first loop would have made it to n(n - 1) however, because they don’t compare twice each time, and the number of comparisons decreases each pass, it is divided by 2.	This program behaves like a bubble sort but is very different because instead of swapping and sorting the array, it searches for a certain value. The first loop is responsible for assigning the first value for a specific index of an array and the second loop for a specific index that will be compared to the first index. Moving to int main, the first loop is responsible for creating random integers and the second loop is responsible for printing out the values inside an array.

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
const int maxVal = 10;

//Algorithm Unique (A)


bool alUnique (int arr[], int A){
    for (int i = 0; i < A; i++){
        for(int j = i + 1; j < A; j++){
            if (arr[i] == arr[j]){
                return false;
            }
        }
    }
    return true;
}

int main(){

    //generate random set
    int set[maxVal];
    srand(time(0));

    for (int i = 0; i < maxVal; i++){
        set[i] = rand() % 10;
    }

    for(int i = 0; i < maxVal; i++){
        std::cout<<set[i]<<" ";
    }

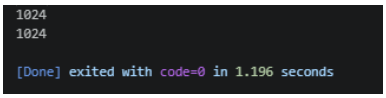
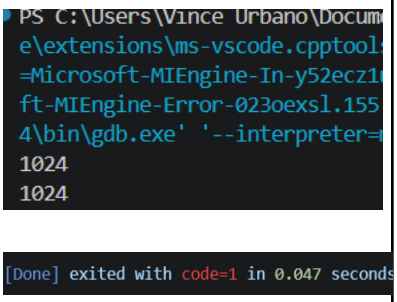
    std::cout<<std::endl;

    std::cout<<alUnique(set, maxVal);

    return 0;
}
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3 y = brpower(x, (n-1)/2)
4 return x*y*y
5 if n is even then
6 y = brpower(x, n/2)
7 return y*y

	Dimaguiba	Urbano
Theoretical Analysis	They both compute their outputs with exponents depending on “n”. Rpower is more straightforward representing a complexity of O(n) as it only decreases n by 1 (n - 1) each time it recurs. brpower does so by halving it each time. Thus giving it a complexity of O(logn) till the condition is reached.	The algorithm of rpower has a time complexity of O(n) because it reduces the exponent by 1 in each step which requires n+1 recursive calls. Brpower on the other hand has a time complexity of O(logn) because it divides the exponent by 2 in each step.
Experimental Analysis		
Analysis and Comparison	Both result in the same conclusion. The only difference is the method they took. Brpower would prove to be more efficient as it halves the number of comparisons there has to be.	After the program runs, we can see that the output is the same for both rpower and brpower. Rpower computes it by multiplying x one at a time but brpower calculates it by using an algebraic rule to repeatedly square powers. In this algorithm, brpower is more efficient since it uses squaring rules instead of multiplying it one by one.

CODE

```cpp
#include <iostream>

int rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

int brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        int y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    int num1 = 2;
    int num2 = 10;


    std::cout << rpower(num1,num2) <<std::endl;
    std::cout << brpower(num1,num2)<<std::endl;

    return 0;
}
```

Conclusions:

Dimaguiba:
Measuring the performance and time and space it takes to run a program may dictate their efficiency in many ways. However, there are several factors such as the strength of hardware. Hardware alone is not the best way to measure and predict the amount of time it takes to run a program. This is due to the fact that the strength of one's hardware may vary. Some faster, some slower. This is where time and space complexities come in. They are the ones that help us measure or estimate the amount of space and time it takes to run a program. Some libraries are introduced like "chrono" to help with that process. Time frequencies and complexities are based on the methods and lines used. Single-line commands tend to only count as one operation, while loops like "for" and "while" are represented through "n" as the number of operations depend on the size of that variable. We can combine these complexities and simplify them through Big-O notation.

Urbano:
In this activity, I have learned more about basic algorithm analysis and how I can use them to understand how my program works and which algorithm is best to use using theoretical analysis. I have also learned new libraries like chrono which can be very useful for experimental analysis. Chrono is very helpful for experimental analysis because it is mostly about the hardware.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

Dimaguiba:
I'd say I did decently, following the procedure and such, I was able to understand that part well. The part where I struggled was the Supplementary activities. I was able to make a working version of the algorithms, but some part of me thinks there's something missing. I was also quite confused on the theoretical analysis parts and still quite am,

with some sources conflicting with each other, and the struggle to find material that explains the process thoroughly without all the jargon.

Urbano:
I think that I did good during the activity but I still have plenty of room for improvement. I think that I can still improve my critical thinking skills when it comes to theoretical Analysis. I think that this is what I need to improve the most since I still find it very confusing when it comes to complex algorithms. During this activity I believe that I have improved a lot comparing to myself before this activity which is why I think that I did good during this activity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Searching for sources about time frequencies and complexities.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.